

CEFET-MG

Engenharia de Computação

Trabalho Final

Estação de Monitoramento de
Temperatura, Umidade e Relógio
com ESP32, DHT11 e Display OLED SSD1306

Disciplina:

Microcontroladores

Aluno(s):

Aléxia Cordeiro Oliveira

Junho de 2026

Sumário

1	Objetivo Geral	2
2	Descrição Detalhada do Projeto	2
2.1	Hardware utilizado	2
2.2	Descrição funcional	2
2.3	Arquitetura do driver do display	3
3	Fluxograma do Projeto	4
3.1	Fluxo principal (setup + loop)	4
3.2	Fluxo da ISR do Timer0	5
3.3	Fluxo da leitura do DHT11	6
4	Diagrama de Estados	7
5	Periféricos Utilizados e Configuração dos SFRs	7
5.1	GPIO — Registradores de Entrada/Saída	7
5.2	Timer0 por Hardware — Relógio de 1 segundo	8
5.2.1	Cálculo do período de 1 segundo	8
5.2.2	Configuração dos registradores	8
5.3	I2C (Wire.h) — Comunicação com o Display SSD1306	8
5.3.1	Protocolo de comunicação com o SSD1306	9
5.4	UART (Serial) — Log serial com timestamp	9
6	Justificativa do Clock do Microcontrolador	9
7	Justificativa dos Delays e Loops de Espera	9
7.1	Delays no protocolo DHT11	9
7.2	Delay de power-on do display	10
7.3	Loops de espera não bloqueantes	10
8	Driver do Display SSD1306 — Detalhamento	11
8.1	Organização da GDDRAM	11
8.2	Sequência de inicialização	11
8.3	Envio do framebuffer (fb_flush)	11
9	Resultados	12
9.1	Funcionamento do display	12
9.2	Log serial	12
9.3	Consumo de recursos	12
10	Conclusão	12
	Referências	13

1 Objetivo Geral

O objetivo deste projeto é implementar uma **estação de monitoramento ambiental** utilizando o microcontrolador ESP32, capaz de:

- Medir temperatura e umidade relativa do ar por meio do sensor DHT11, controlado diretamente via registradores GPIO — sem uso de biblioteca de alto nível;
- Manter um relógio (HH:MM:SS) de alta precisão utilizando o *hardware timer* interno do ESP32 e rotina de serviço de interrupção (ISR);
- Exibir as informações em tempo real em um display OLED SSD1306 128×64 pixels, controlado por driver próprio via barramento I2C — sem uso da biblioteca Adafruit;
- Registrar leituras com *timestamp* na porta serial (UART).

Todos os periféricos internos do ESP32 são configurados diretamente via registradores de funções especiais (SFRs), conforme exigido pela disciplina. A comunicação serial (I2C e UART) utiliza as bibliotecas do compilador (`Wire.h` e `Serial`), devidamente justificadas.

2 Descrição Detalhada do Projeto

2.1 Hardware utilizado

Tabela 1: Componentes do projeto

Componente	Modelo	Conexão
Microcontrolador	ESP32 DevKit V1	—
Sensor de temp./umidade	DHT11	GPIO4
Display	OLED SSD1306 128×64	I2C (SDA=21, SCL=22)
LED indicador	LED interno	GPIO2

2.2 Descrição funcional

O sistema opera em três tarefas concorrentes não bloqueantes, coordenadas por temporização via `millis()`:

1. **Relógio por hardware (ISR):** O Timer0 do ESP32 é configurado com prescaler 80, gerando uma base de tempo de 1 MHz. O alarme é programado para 1.000.000 ciclos, disparando a ISR `onTimer()` a cada exatamente 1 segundo para incrementar segundos, minutos e horas.
2. **Leitura do DHT11 (a cada 2,5 s):** O protocolo do DHT11 é implementado diretamente via registradores GPIO (`GPIO.enable_w1ts`, `GPIO.out_w1tc`, `GPIO.in`). A captura de tempo dos 40 bits usa `esp_timer_get_time()` com resolução de 1 μ s. O LED interno acende durante a leitura como indicador visual.
3. **Atualização do display (a cada 500 ms):** Um framebuffer de 1024 bytes é construído em RAM com todas as informações (relógio, temperatura, umidade e status) e enviado ao display SSD1306 em uma única sequência I2C, eliminando flickering.

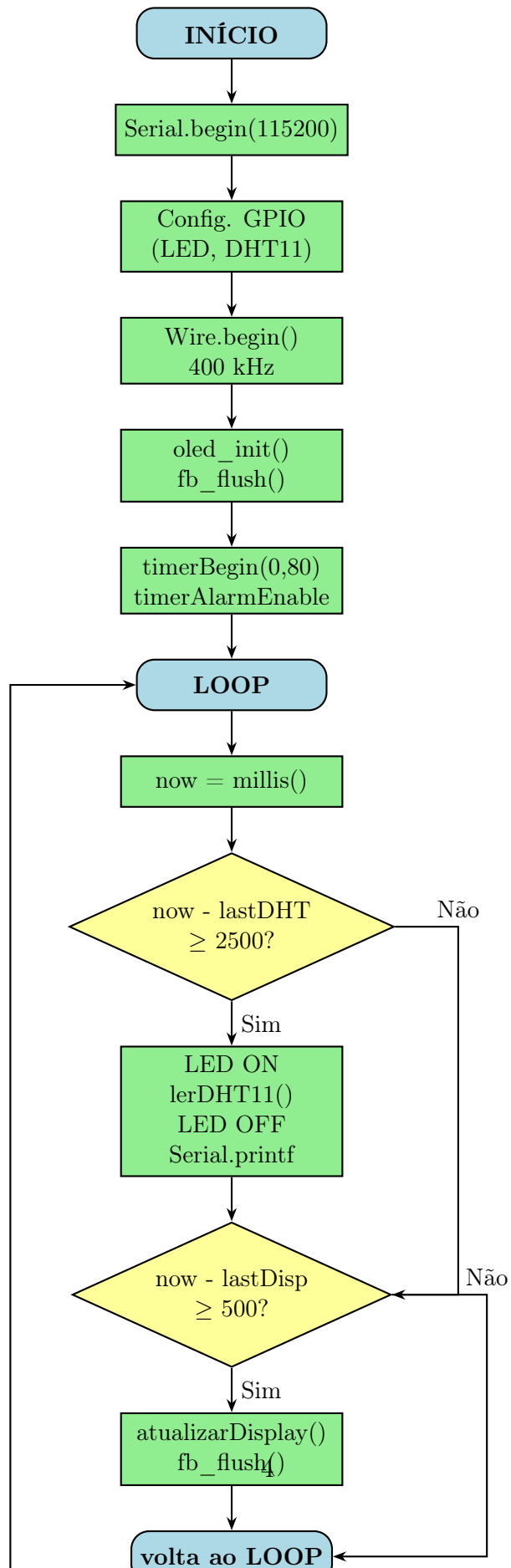
2.3 Arquitetura do driver do display

O driver do SSD1306 foi implementado em três camadas:

- **Camada de transporte:** funções `oled_cmd()` para comandos e `fb_flush()` para dados, usando `Wire.h`.
- **Camada de framebuffer:** array `fb[1024]` em RAM representando a GDDRAM do display (8 páginas $\times$ 128 colunas). Funções `fb_pixel()`, `fb_hline()`, `fb_puts()`, `fb_puts2x()` desenharam no buffer sem I2C.
- **Fonte bitmap:** tabela `font5x7` (ASCII 0x20–0x7E) armazenada em Flash (`PROGMEM`), substituindo o `Adafruit_GFX`. Os arquivos `glcdfont.c` e `Adafruit_GFX.cpp`, da biblioteca `Adafruit-GFX`, foram usados como referência: o primeiro para a matriz dos caracteres da fonte 5 $\times$ 7 e o segundo para a lógica de varredura das colunas e escrita dos pixels. Ambos foram reimplementados de forma simplificada no driver próprio, sem uso direto da biblioteca.

3 Fluxograma do Projeto

3.1 Fluxo principal (setup + loop)



3.2 Fluxo da ISR do Timer0

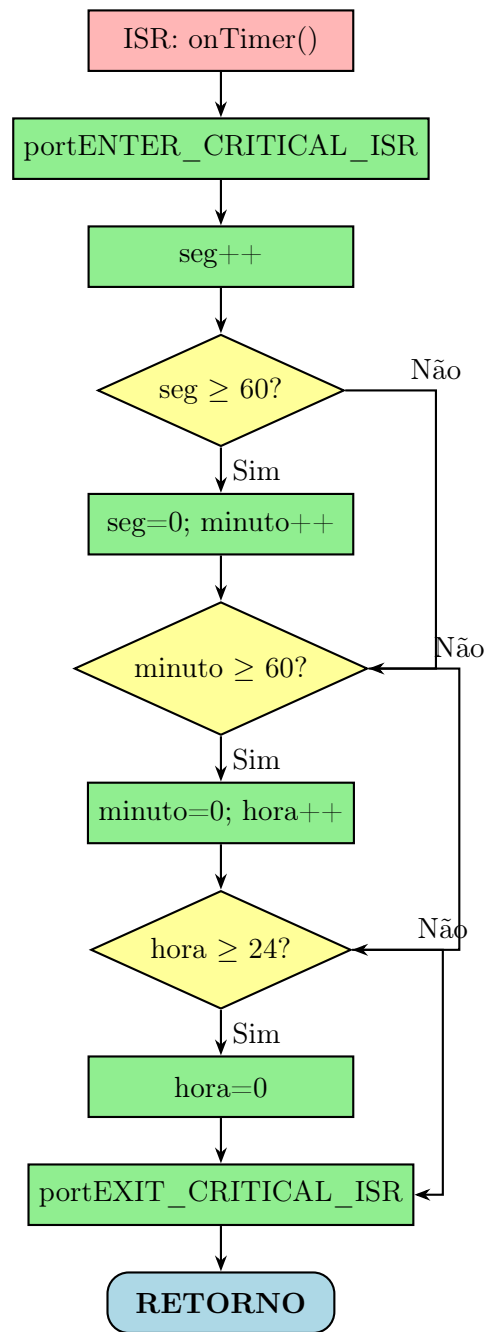


Figura 2: Fluxograma da ISR do Timer0 (relógio)

3.3 Fluxo da leitura do DHT11

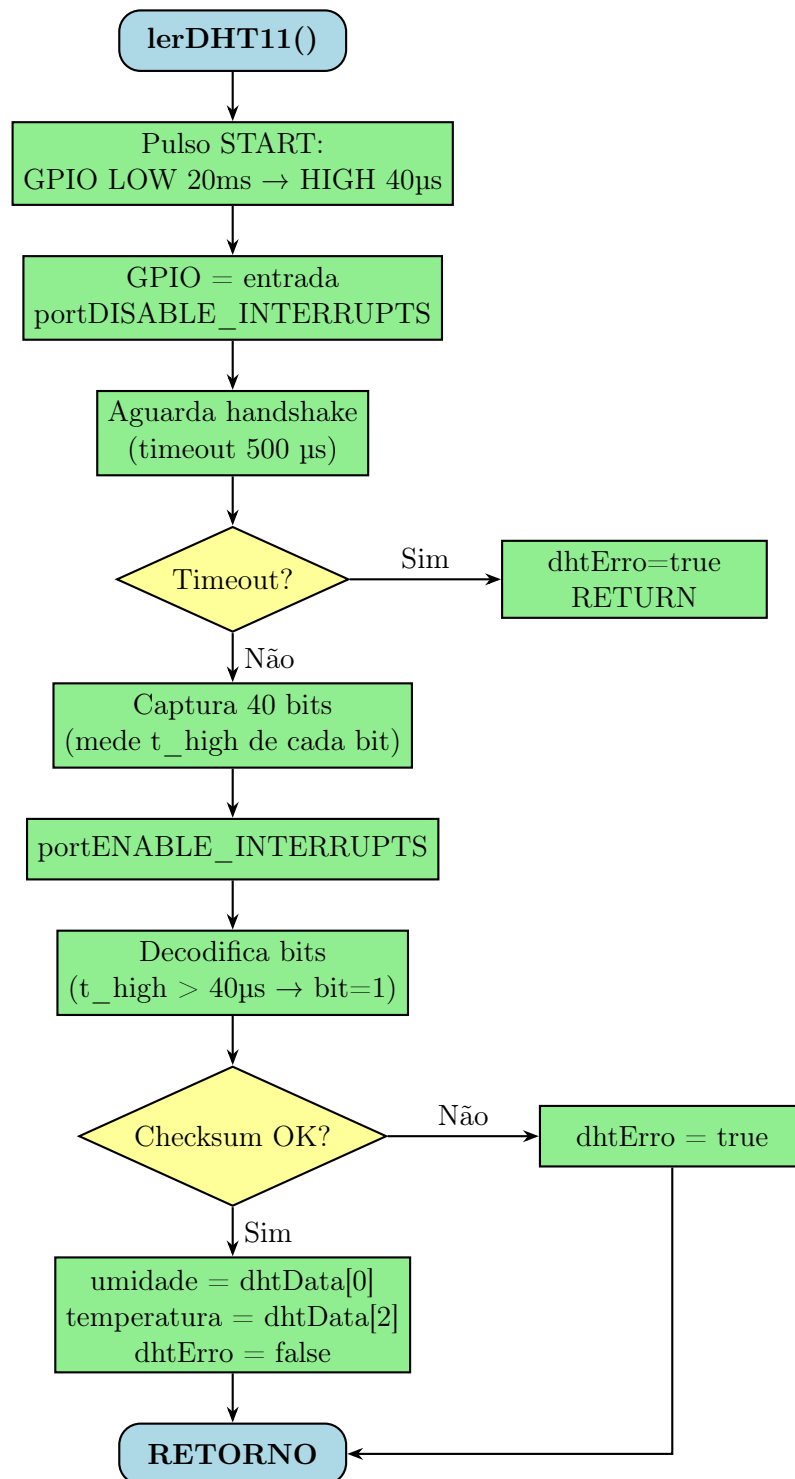


Figura 3: Fluxograma da rotina de leitura do DHT11

4 Diagrama de Estados

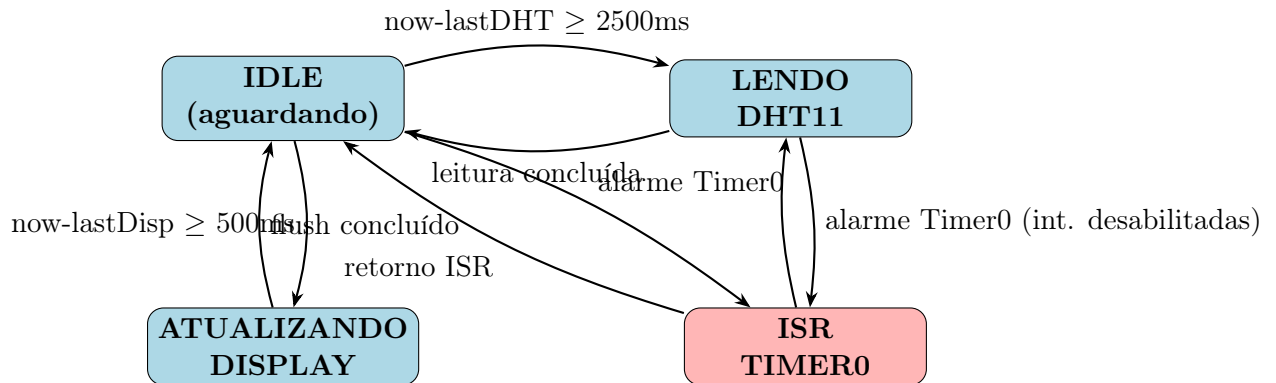


Figura 4: Diagrama de estados do sistema

Observação importante: durante a captura dos 40 bits do DHT11 (*bit-bang* de protocolo sensível ao tempo), as interrupções são desabilitadas com `portDISABLE_INTERRUPTS()` para garantir a precisão das medições de temporização. Por isso, o Timer0 não pode disparar a ISR nesse intervalo (tipicamente <4 ms). Assim que a captura termina, as interrupções são reabilitadas com `portENABLE_INTERRUPTS()`.

5 Periféricos Utilizados e Configuração dos SFRs

5.1 GPIO — Registradores de Entrada/Saída

O ESP32 possui registradores do tipo *write-1-to-set* e *write-1-to-clear* para operações atômicas sem read-modify-write, evitando condições de corrida.

Tabela 2: Registradores GPIO utilizados

Registrador	Função	Uso no projeto
<code>GPIO.enable_w1ts</code>	Habilita pino como saída (W1S)	LED (GPIO2), DHT11 saída (GPIO4)
<code>GPIO.enable_w1tc</code>	Desabilita saída / configura entrada (W1C)	DHT11 entrada (GPIO4)
<code>GPIO.out_w1ts</code>	Coloca pino em nível alto (W1S)	LED ON, DHT11 HIGH
<code>GPIO.out_w1tc</code>	Coloca pino em nível baixo (W1C)	LED OFF, DHT11 LOW
<code>GPIO.in</code>	Leitura do estado dos pinos (bits 0–31)	Leitura do bit DHT11

Leitura do pino DHT11:

```

1 // Leitura do bit 4 do registrador de entrada
2 uint8_t nivel = (GPIO.in >> DHT11_PIN) & 1;

```


5.2 Timer0 por Hardware — Relógio de 1 segundo

O ESP32 possui 4 timers de 64 bits de uso geral (*General Purpose Timers*). O Timer0 é configurado como contador crescente com prescaler e alarme recorrente.

5.2.1 Cálculo do período de 1 segundo

$$f_{\text{APB}} = 80 \text{ MHz} \quad (1)$$

$$\text{Prescaler} = 80 \quad (2)$$

$$f_{\text{timer}} = \frac{f_{\text{APB}}}{\text{Prescaler}} = \frac{80 \text{ MHz}}{80} = 1 \text{ MHz} \quad (3)$$

$$T_{\text{timer}} = \frac{1}{1 \text{ MHz}} = 1 \mu\text{s} \quad (4)$$

$$N_{\text{alarme}} = \frac{1 \text{ s}}{1 \mu\text{s}} = 1.000.000 \text{ ciclos} \quad (5)$$

5.2.2 Configuração dos registradores

Tabela 3: Configuração do Timer0

Parâmetro	Valor	Significado
Timer selecionado	0	Timer0 (0 a 3 disponíveis)
Prescaler	80	Divide clock APB de 80 MHz por 80
Sentido	true (crescente)	Conta de 0 até o alarme
Valor do alarme	1.000.000	Período de exatamente 1 s
Auto-reload	true	Reinicia automaticamente (recorrente)
ISR	onTimer()	Seção crítica com mutex de spinlock

```

1 timer0 = timerBegin(0, 80, true);
2 timerAttachInterrupt(timer0, &onTimer, true);
3 timerAlarmWrite(timer0, 1000000, true);
4 timerAlarmEnable(timer0);

```

Listing 1: Configuração do Timer0

5.3 I2C (Wire.h) — Comunicação com o Display SSD1306

A biblioteca `Wire.h` é a biblioteca de comunicação serial I2C do compilador Arduino/ESP32, equivalente ao uso de `Serial` para UART. Ela é utilizada por ser a única forma portátil de acessar o periférico I2C do ESP32 (registradores `I2C_SCL_LOW_PERIOD_REG`, `I2C_COMMAND_REG`, etc.) sem escrever um driver I2C completo do zero.

Tabela 4: Parâmetros do barramento I2C

Parâmetro	Valor	Justificativa
Pino SDA	GPIO21	Padrão do ESP32 DevKit
Pino SCL	GPIO22	Padrão do ESP32 DevKit
Clock	400 kHz	Fast mode; SSD1306 suporta até 400 kHz
Endereço	0x3C	SA0 do SSD1306 ligado ao GND

5.3.1 Protocolo de comunicação com o SSD1306

O SSD1306 distingue comandos de dados pelo *byte de controle* enviado logo após o endereço I2C:

- 0x00: byte(s) seguinte(s) são **comandos** de controle do display (modo, contraste, endereçamento, etc.);
- 0x40: byte(s) seguinte(s) são **dados** da GDDRAM (pixels).

5.4 UART (Serial) — Log serial com timestamp

Tabela 5: Parâmetros da UART

Parâmetro	Valor	Justificativa
Baud rate	115200	Taxa padrão dos conversores USB-Serial do DevKit
Formato	8N1	8 bits dados, sem paridade, 1 stop bit (padrão universal)

6 Justificativa do Clock do Microcontrolador

O ESP32 opera com clock de CPU de **240 MHz** (modo padrão do ESP-IDF/Arduino). O clock do barramento APB (*Advanced Peripheral Bus*), que alimenta os periféricos (timers, I2C, UART), é fixo em **80 MHz**.

Tabela 6: Clocks do sistema

Clock	Valor	Alimenta
CPU	240 MHz	Núcleos Xtensa LX6
APB	80 MHz	Timers, I2C, UART, SPI
Timer0 (pós-prescaler)	1 MHz	ISR do relógio
I2C	400 kHz	Comunicação com SSD1306
UART	115200 baud	Log serial

7 Justificativa dos Delays e Loops de Espera

7.1 Delays no protocolo DHT11

O DHT11 possui um protocolo de comunicação *single-wire* com temporização rígida, definida pelo seu datasheet:

Tabela 7: Delays utilizados na comunicação com o DHT11

Delay	Valor	Justificativa
Pulso LOW de Start	<code>delay(20)</code> = 20 ms	Datasheet exige mínimo de 18 ms para o host sinalizar início de comunicação. Usado antes de desabilitar interrupções, pois <code>delay()</code> depende do Sys-Tick.
Pulso HIGH de Start	<code>delayMicroseconds(40)</code> = 40 μ s	Tempo de liberação do barramento antes de configurar o pino como entrada. O DHT11 responde após 20–40 μ s.
Limiar bit 0/1	$t_{\text{high}} > 40 \mu\text{s}$	Bit 0: pulso HIGH de 26–28 μ s; Bit 1: pulso HIGH de 70 μ s. O limiar de 40 μ s distingue os dois com margem segura.
Timeout dos loops	500 μ s	Valor conservador acima do máximo esperado nos pulsos do DHT11 (80 μ s), garantindo detecção de falha sem travar o sistema.

Além dos atrasos fixos do pulso de START, a rotina usa laços `while` para aguardar as transições de nível lógico do DHT11. Esses loops são necessários porque o sensor não responde em instantes exatamente fixos: o programa precisa esperar a subida e a descida de cada pulso para medir sua duração real. Cada loop possui timeout de 500 μ s para substituir esperas bloqueantes indefinidas; assim, se o sensor estiver desconectado, com erro de checksum ou preso em um nível lógico, a função retorna com `dhtErro=true` sem travar o sistema.

7.2 Delay de power-on do display

```
1 delay(100); // aguarda estabilizacao do Vcc do display apos power-on
```

O SSD1306 necessita de pelo menos 5 ms para estabilizar a tensão de alimentação após o power-on antes de aceitar comandos I2C. O valor de 100 ms é conservador e garante inicialização confiável em qualquer condição de fonte de alimentação.

7.3 Loops de espera não bloqueantes

No `loop()` principal, os intervalos de tempo são gerenciados por:

```
1 if (now - lastDHTRead >= DHT_INTERVAL) { ... } // 2500 ms
2 if (now - lastDisplayUpd >= DISP_INTERVAL) { ... } // 500 ms
```

Essa técnica (*millis-based scheduling*) evita o uso de `delay()` no loop principal, mantendo o sistema responsivo e permitindo que múltiplas tarefas executem concorrentemente de forma cooperativa.

8 Driver do Display SSD1306 — Detalhamento

8.1 Organização da GDDRAM

A memória de display (GDDRAM) do SSD1306 possui 128×64 bits, organizados em 8 *páginas* de 8 linhas cada:

$$\text{Índice no framebuffer} = \left\lfloor \frac{y}{8} \right\rfloor \times 128 + x \quad \text{Bit} = y \bmod 8 \quad (6)$$

8.2 Sequência de inicialização

Tabela 8: Comandos de inicialização do SSD1306

Comando	Valor	Função
0xAE	—	Display OFF
0xD5, 0x80	divisor=1, fosc=8	Clock interno padrão
0xA8, 0x3F	63	Mux ratio: 64 linhas
0xD3, 0x00	0	Display offset = 0
0x40	—	Start line = 0
0x8D, 0x14	habilitado	Charge pump (Vcc interno)
0x20, 0x00	horizontal	Modo de endereçamento
0xA1	—	Segment remap
0xC8	—	COM scan direction remapped
0xDA, 0x12	alternativo	COM pins config
0x81, 0xCF	0xCF	Contraste
0xD9, 0xF1	fase1=1, fase2=15	Pre-charge period
0xDB, 0x40	$0,89 \times V_{cc}$	Vcomh deselect level
0xA4	—	Exibe GDDRAM
0xA6	—	Modo normal (não invertido)
0xAF	—	Display ON

8.3 Envio do framebuffer (fb_flush)

O framebuffer de 1024 bytes é enviado em 32 transações I2C de 32 bytes cada (limite do buffer interno do Wire.h), precedidas de uma transação de comandos configurando a janela de endereçamento:

```

1 static void fb_flush() {
2     // Configura janela completa: colunas 0-127, paginas 0-7
3     Wire.beginTransaction(SSD1306_ADDR);
4     Wire.write(0x00);
5     Wire.write(0x21); Wire.write(0x00); Wire.write(0x7F);
6     Wire.write(0x22); Wire.write(0x00); Wire.write(0x07);
7     Wire.endTransmission();
8
9     // Envia 1024 bytes em blocos de 32
10    for (uint16_t i = 0; i < 1024; i += 32) {
11        Wire.beginTransaction(SSD1306_ADDR);
12        Wire.write(0x40);           // controle: dados GDDRAM

```

```

13     Wire.write(&fb[i], 32);
14     Wire.endTransmission();
15 }
16 }

```

Listing 2: Função fb_flush()

9 Resultados

9.1 Funcionamento do display

O display exibe corretamente as seguintes informações:

- **Linha 1 (y=0):** Relógio no formato HH:MM:SS, atualizado a cada 1 segundo pela ISR do Timer0.
- **Divisor (y=9):** Linha horizontal de separação.
- **Linhas 2-3 (y=12 a 45):** Temperatura (T:XX.XC) e umidade (U:XX.X%) em fonte 2x, com espaçamento adequado.
- **Divisor (y=54):** Linha horizontal de separação inferior.
- **Status (y=56):** Status: OK ou Status: ERRO.

Em caso de falha na leitura do DHT11 (checksum incorreto ou timeout), o sistema exibe Erro DHT11 e Verificar conexao, enquanto o relógio e o status continuam funcionando normalmente.

9.2 Log serial

A cada 2,5 segundos, uma linha é impressa na porta serial com timestamp:

```

[00:00:02] Temp: 24.0C | Umid: 65.0%
[00:00:05] Temp: 24.0C | Umid: 65.0%
[00:00:07] ERRO na leitura do DHT11

```

9.3 Consumo de recursos

Tabela 9: Uso de memória estimado

Recurso	Tamanho	Local
Framebuffer fb[1024]	1024 bytes	RAM (DRAM)
Fonte font5x7	$95 \times 5 = 475$ bytes	Flash (PROGMEM)
Variáveis do DHT11	≈ 20 bytes	RAM
Stack da ISR	≈ 64 bytes	RAM

10 Conclusão

O projeto implementou com sucesso uma estação de monitoramento ambiental completa utilizando o ESP32. Todos os requisitos da disciplina foram atendidos:

- **Sem bibliotecas de alto nível para periféricos:** GPIO e DHT11 controlados por registradores; display SSD1306 por driver próprio.
- **Bibliotecas seriais justificadas:** `Wire.h` (I2C) e `Serial` (UART), com parâmetros documentados.
- **Display para comunicação:** OLED SSD1306 com driver e fonte bitmap próprios.
- **Quatro periféricos internos:** GPIO, Timer0, I2C e UART.
- **Interrupção por hardware:** `ISR onTimer()` com seção crítica por `spinlock`.

A técnica de *framebuffer* para o display eliminou o flickering que ocorria com envios I2C fragmentados, e o uso de temporização não bloqueante (`millis()`) garantiu a execução concorrente e confiável das três tarefas do sistema.

Referências

1. Solomon Systech. *SSD1306 — 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Rev 1.1, 2008.
2. Espressif Systems. *ESP32 Technical Reference Manual*, v5.3. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
3. Aosong Electronics. *DHT11 — Temperature & Humidity Sensor Product Manual*, v3.
4. Arduino. *Wire Library Documentation*. Disponível em: <https://www.arduino.cc/en/Reference/Wire>
5. Adafruit. *Adafruit-GFX-Library — glcdfont.c*. Disponível em: <https://github.com/adafruit/Adafruit-GFX-Library/blob/master/glcdfont.c>
6. Adafruit. *Adafruit-GFX-Library — Adafruit_GFX.cpp*. Disponível em: https://github.com/adafruit/Adafruit-GFX-Library/blob/master/Adafruit_GFX.cpp